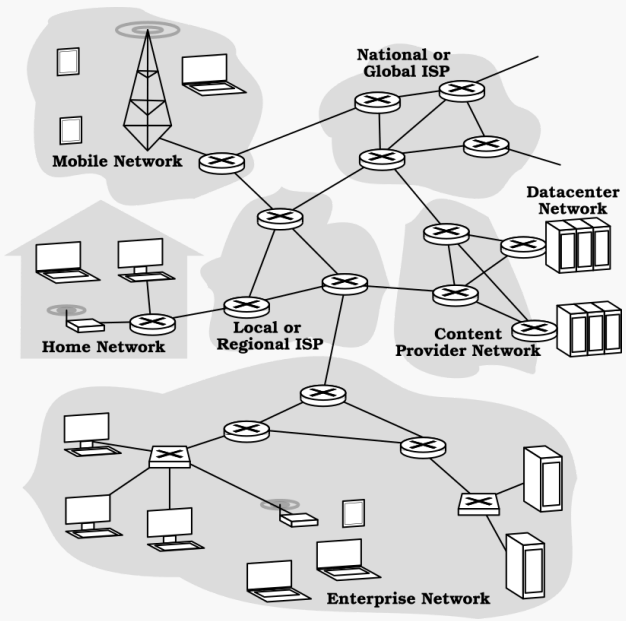
13 - Graphs

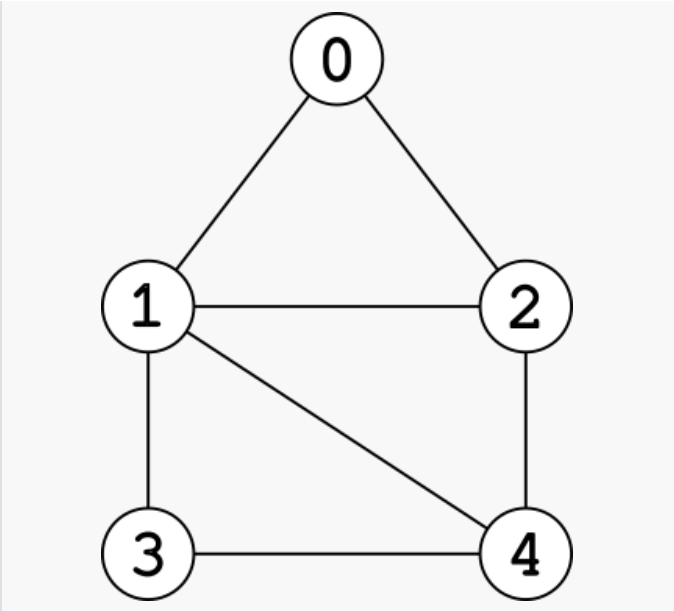
A **graph** G is defined by $G = (V, E)$ where V represents the set of vertices and E represents the set of edges that compose G. A **vertex** is a node (element) within the graph. An **edge** connects (associates) two vertices in the graph such that $e_{ij} = (v_i, v_j)$ where $V \ni v_i, v_j$ and $E \ni e_{ij}$. We encounter graphs everywhere in daily life; although, systems that can be organized as graphs may not always take a clear form as a graph.

Washington DC Metro Subway Map

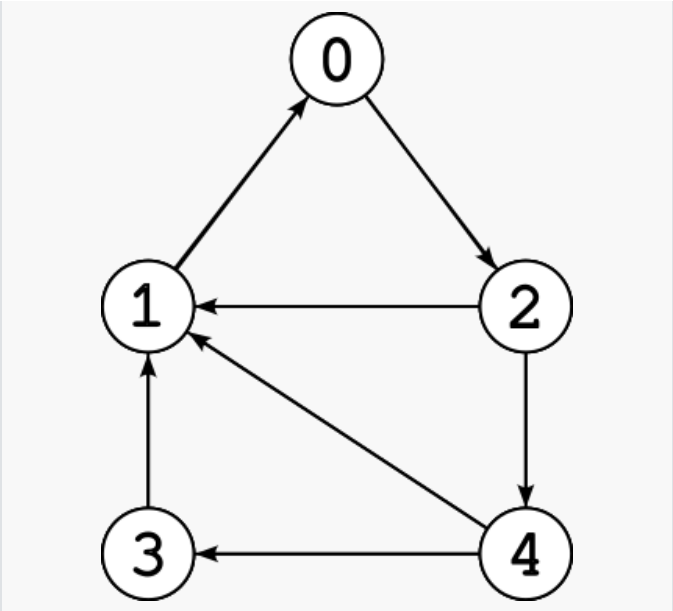


Internet Topology

Two vertices are **adjacent** if they share an edge joining together and an edge may be **directed** or **undirected**. A **directed graph** is a graph that contains directed edges while an **undirected graph** is a graph that does not contain any directed edges.



Undirected



Directed

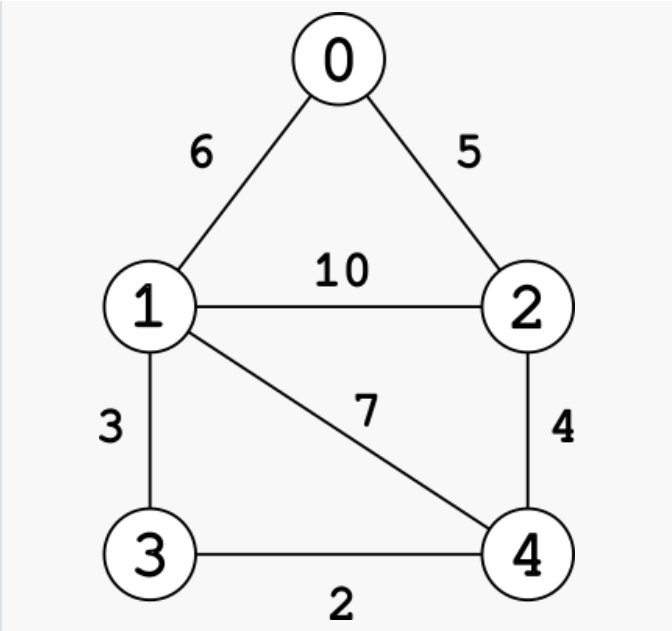
An **undirected edge** does not suggest or create any directional constraints. An undirected edge is typically represented by just a line segment between two vertices and can be traversed in either direction along the edge's axis. In an undirected edge, the edge association (v_i, v_j) is transitive or in other words, $e_{ij} = e_{ji}$, so there is no need to consider the order in the vertex pair and the vertices can appear in any order in the pair.

A **directed edge** points in a specified direction. A directed edge is typically represented by an arrow that suggests transitions are only allowed in the direction in which the edge points. The tail of the edge indicates the vertex at which the edge originates while the arrowhead indicates the vertex at which the edge terminates. For the directed edge $e_{ij} = (v_i, v_j)$, the order of the vertices matters, so the first vertex in the pair, v_i, designates the originating vertex and the second vertex, v_j, designates the terminating vertex. Note that this means that the edge association is not transitive for a directed graph so all directed edges must be explicitly defined. Alternative syntax might include an overarrow pointing from origin to terminator, $e_{ij} = \overrightarrow{(v_i, vj)}$, to specify direction.

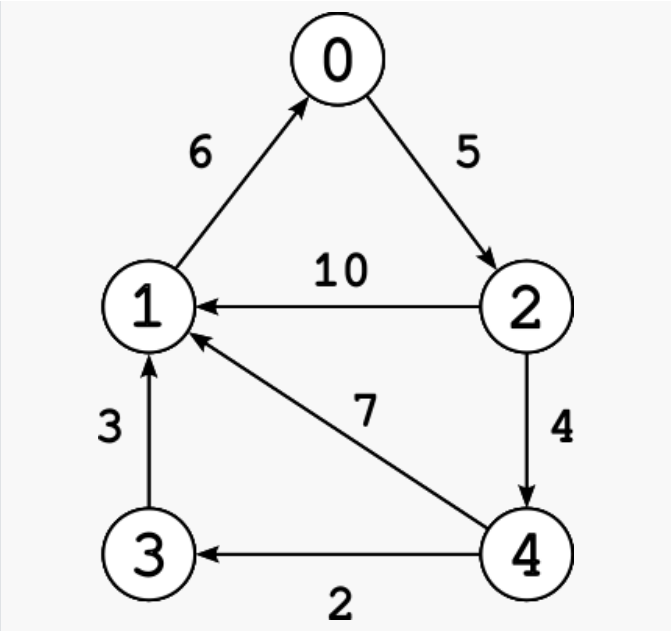
For vertices in a directed graph we differentiate **outgoing edges** and **incoming edges**. Outgoing edges are defined by where the tails of any edges drawn with an arrow emerge from a vertex and incoming edges are defined by the node that is pointed to by the arrow head at the end of an edge. In an undirected graph, all edge terminators define both incoming and outgoing

The **degree** of a vertex is the measure of the number of edges connected to it. Degree may be further defined for a directed graph where degree is subdivided into **indegree** and **outdegree**. The **indegree** is the number of incoming edges to the vertex and the **outdegree** is the number of outgoing edges from the vertex. For an undirected graph where there is no discrimination between incoming and outgoing edges, so the indegree and outdegree are equal and is generally summarized simply as degree. For a directed graph, indegree and outdegree are not necessarily equal and these metrics are maintained independent of one another.

Edges may have a value associated with them which is called the edge **weight** and a graph with weighted edges is a **weighted graph**. A weighted graph can be directed or undirected.



Undirected and Weighted

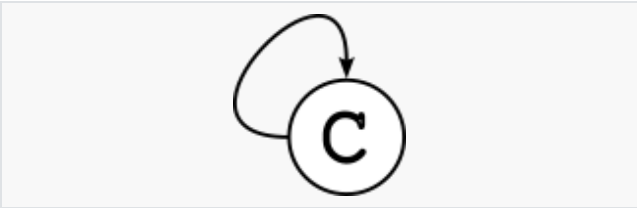


Directed and Weighted

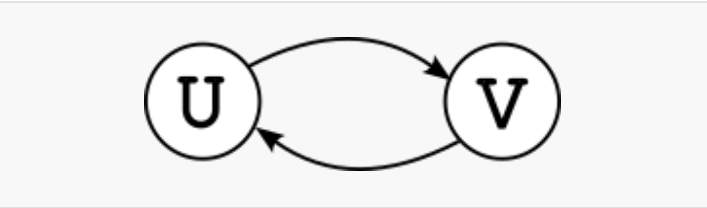
Weights represent a **cost metric** associated with the transition along that edge. Weight is intentionally abstract as it may represent a variety of costs such as a measure of distance, time, money, energy to traverse along that edge. For this reason weights may also be called costs. In an unweighted graph, we can assume that all the weight of each edge is generally equal to 1, or in other words, the cost of traveling along any edge is equal.

Two vertices, v_i and v_j, are **connected** if there exists a set of edges between them that enables transition from v_i and v_j. If v_i and v_j are connected, the set of vertices and edges that connect v_i to v_j form a **path** between the two edges. For a directed graph, a directed path adhere to any direction constraints.

A **cycle** is a configuration of edges and vertices that results in a path back to an originating vertex. In other words a cycle exists if there exists a path from v_i to v_i. Cycles may be formed by a vertex with an outgoing edge that is also an ingoing edge to that vertex or by a sequence of vertices that begins and ends with the same vertex. A graph with no cycles is **acyclic**.



A single vertex can form a cycle



Multiple vertices can form a cycle

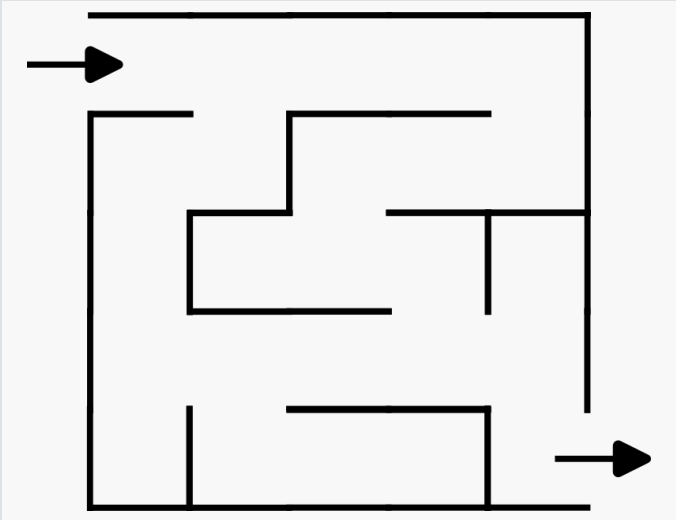
If v_i and v_j are connected, **distance** is the **total cost** of the path from v_i to v_j. For an unweighted graph, the total cost is the number of edges in the path connecting v_i and v_j. For a weighted graph, the total cost is the sum of the weights of all edges in the path from v_i to v_j. The **shortest path** from v_i to v_j is the path with the lowest cost.

Graphs are everywhere and we can represent every data structure we have so far studied as a graph. For example, any linked list meets the definition of a graph:

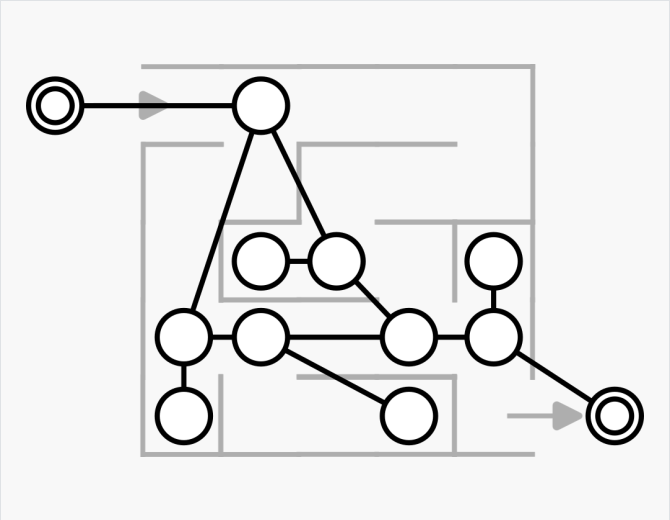
- All linked-lists are composed of vertices and edges where vertices are linked list nodes and edges are the references between nodes.
- All linked-lists can be considered to be directed graphs, whether singly linked or double linked because references create direction.
- The linking of nodes in a linked-list constructs a path that links the head and the tail and all nodes in a linked-list are connected as a result.
- Further, if the linked list is non-circular and singly-linked, it contains no cycles and is acyclic, but circular linked-lists and doubly-linked lists are cyclic.

Furthermore, there are many problems we encountered in the class that can be expressed as a graph. For example, the maze we studied with backtracking can be reimagined as a graph:

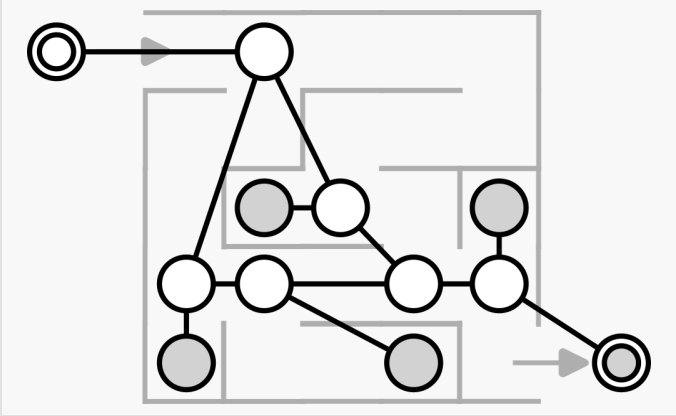
- Each cell in the maze can be described by the number of walls it has. A square cell has 0, 1, 2, or 3 walls. Two cells are adjacent if there is no wall between them.
- Assume that the start and end locations are vertices. If traversal begins at the start vertex and the end vertex is ever reached during graph exploration, there exists a path in the graph from start to end.
- A cell with 0, 1, or 3 walls represents a decision point, whether it is a choice among multiple available paths or a point at which backtracking is necessary. Decision points are graph vertices.
- A cell with only 2 walls can be thought of as a "hall" and only serves to connect decision points. Generally once one travels along a hall, they continue on until they reach the next decision point. Therefore, halls are graph edges. A weight for each edge can be measured by the length of the hall in terms of the number of cells along that hall before a decision point is reached.
- A decision point with 3 walls is a "dead end" and has a degree equal to 1. Backtracking is necessary to escape a dead-end.
- A decision point with 0 or 1 wall is an "intersection" and there are multiple potential paths that emerge from an intersection.



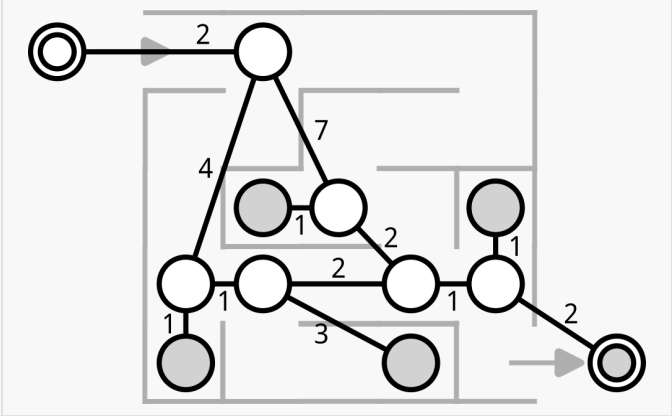
The geometry of a maze



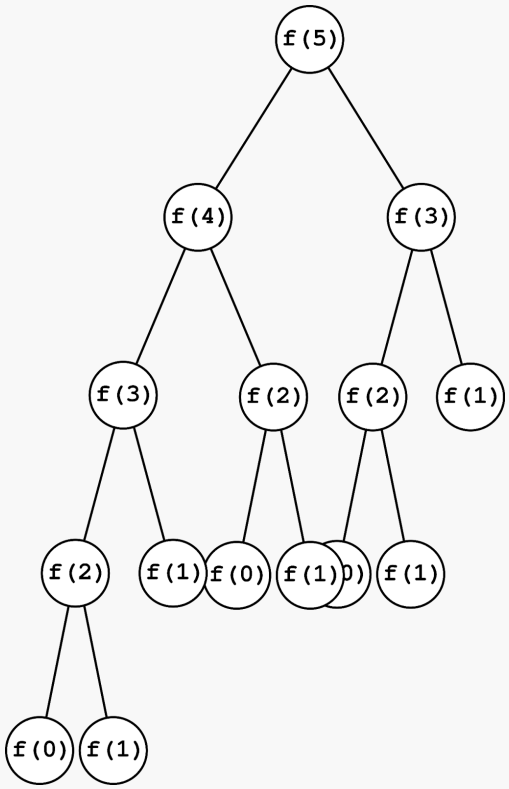
A maze as a graph



A maze demaring terminals and branches



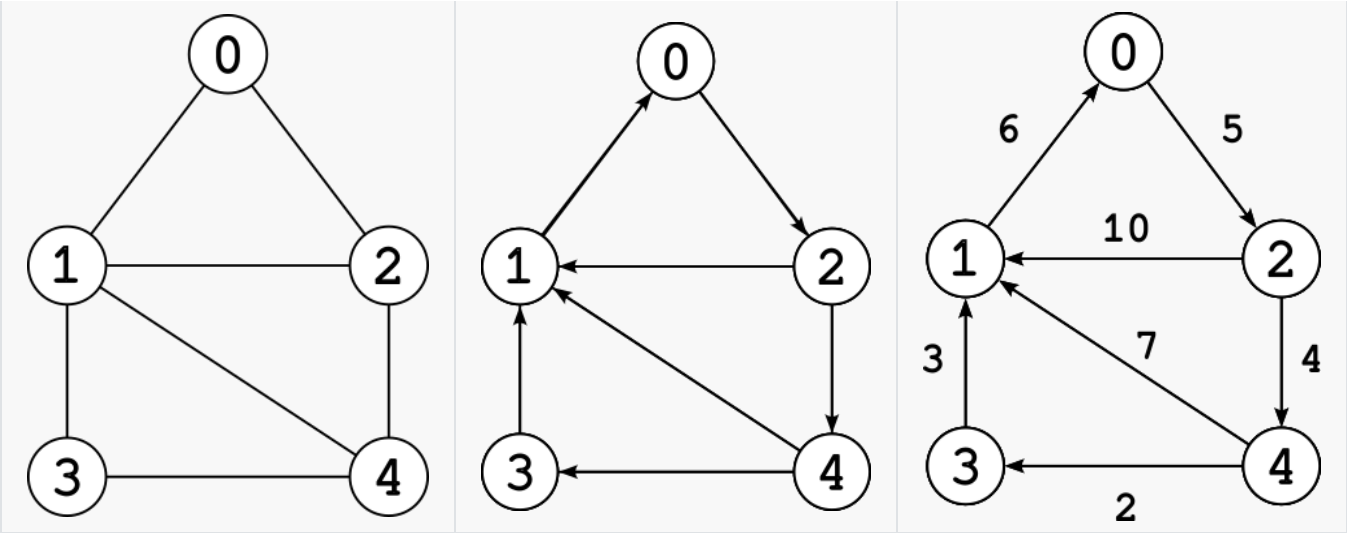
A maze as a weighted graph



A tree (therefore a graph) representing all the recursive calls to fibonacci given an input value of 5 when using a naive recursive implementation

Implementation

The question of how to implement a graph poses the question “given a graph, how can we represent it?”

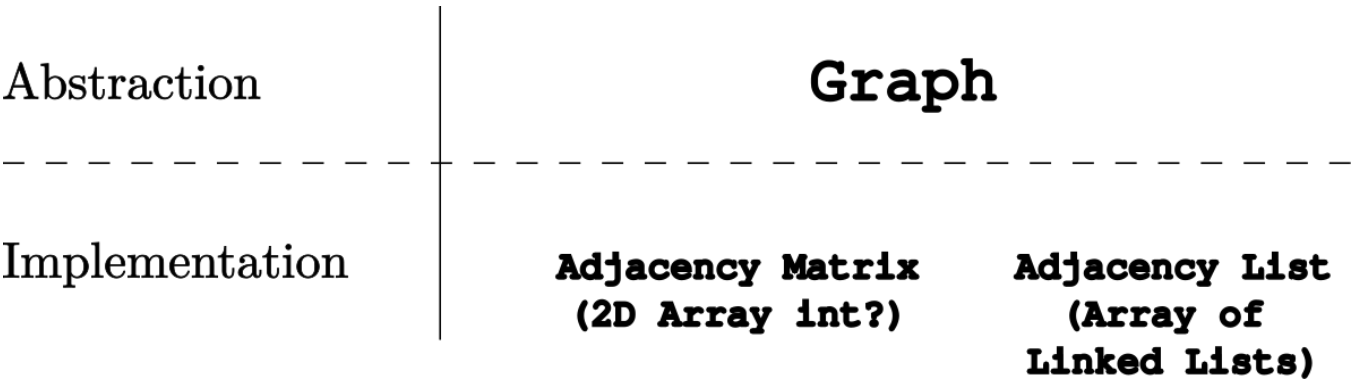


Undirected	*Directed*	*Directed and Weighted*

Given OOP languages, our initial strategy might be to try to use an object oriented, referential approach using Vertex objects and Edge objects. While this is a feasible structure that is capable of representing the graph, it is also difficult to perform efficient queries over the span of the graph as the edge data is not summarized in a succinct, efficient to query form.

Given a purely OOP representational model, our graph has no single summary of adjacency that we can look up and instead we have to traverse the graph to answer a basic question such as whether or not two vertices are adjacent. Recall that two vertices are adjacent if there is an edge connecting the two vertices. If we have to search the graph to fulfill the most fundamental queries that we are needed for more complex operations, we inherit at the very least an $O(n)$ efficiency cost into the foundation of every graph query.

Graphs predate OOP, so we can lean on the basic and very efficient representations that facilitated graphs before OOP such as the adjacency matrix or adjacency list. The adjacency matrix and adjacency list consolidate all of these adjacency relationships into a single data structure for easy access and fast query.

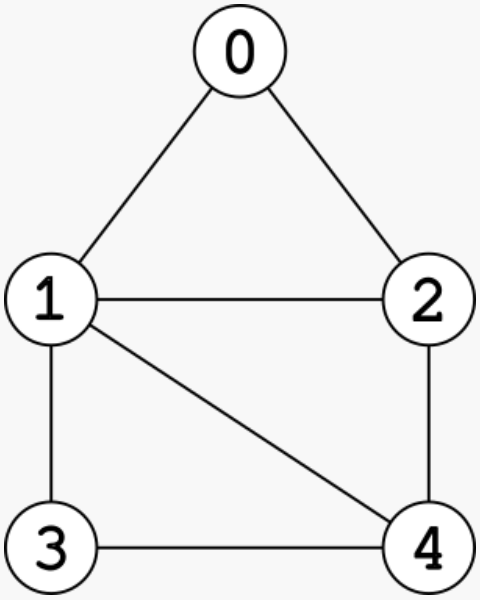


Adjacency Matrix

An **adjacency matrix**, M, uses a two dimensional array to consolidate edge adjacency into a single table. The row and column indices map to one of the vertices in the graph and therefore the matrix is of size $n \times n$ where n is the number of vertices. To determine whether or not two vertices with mapped indices i and j are adjacent, v_i can be queried against v_j simply by looking up the value in $M(i, j)$. For undirected graphs, the matrix is

symmetrical and the property $M(i,j) = M(j,i)$ holds, but this is not the case for directed graphs. In an unweighted graph, the value in the cell will be in the range $[0,1]$ which indicates whether or not the edge (v_i, v_j) exists where a 1 indicates that v_i and v_j are adjacent and there exists an edge joining v_i and v_j while a 0 indicates that v_i and v_j are not adjacent and there is no edge between v_i and v_j. For weighted graphs, the weight of the edge will be stored in the cell and the cell's range is can be any numeric value produced by the cost function associated with the graph.

For example, suppose we have an undirected graph with 5 vertices, then the adjacency matrix will be 5×5 and there will be symmetry such that we can query either by row, column or column, row to determine adjacency.



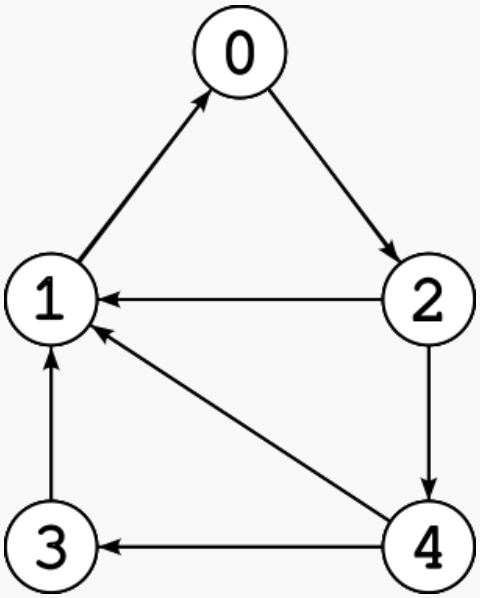
Undirected

	0	1	2	3	4
0	0	1	1	0	0
1	1	0	1	1	1
2	1	1	0	0	1
3	0	1	0	0	1
4	0	1	1	1	0

Adjacency Matrix

As suggested, the adjacency matrix in a directed graph is not symmetrical. This means that $M(i,j) \neq M(j,i)$. In this case, the adjacency matrix designer must clearly document whether row indices map to outgoing edges and column indices map to incoming edges or vice versa.

For example, given the following directed graph example, associated adjacency matrix, and the convention $M(i,j)$, there is an edge (v_0, v_2) because there is a 1 in $M(0,2)$ where i represents as the row index for M and j represents the column index for M. Whether or not two vertices are adjacent is therefore a constant time operation if we know both vertex indices.

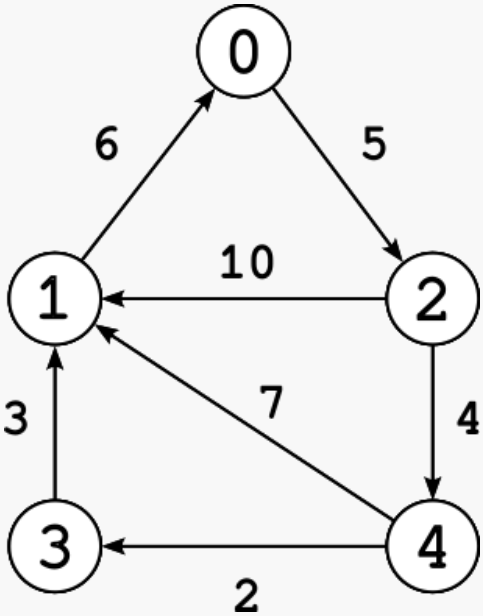


	0	1	2	3	4
0	0	0	1	0	0
1	1	0	0	0	0
2	0	1	0	0	1
3	0	1	0	0	0
4	0	1	0	1	0

Directed | *Adjacency Matrix*

Additionally, it takes linear time in the number of vertices to determine that there is one and only one incoming edge into v_2 because all the remaining cells in column 0 equal 0 and there is only a single 1, in column v_2, on row 0. We can also quickly see that there are two outgoing edges from v_2 because row 2 has two cells containing 1 while all the rest of the cells on row 2 contain 0, indicating no edge.

In order to maintain weights for weighted graphs, the weights for each edge can be stored in the cell instead of using the simple binary indicator.



	0	1	2	3	4
0	0	0	5	0	0
1	6	0	0	0	0
2	0	10	0	0	4
3	0	3	0	0	0
4	0	7	0	2	0

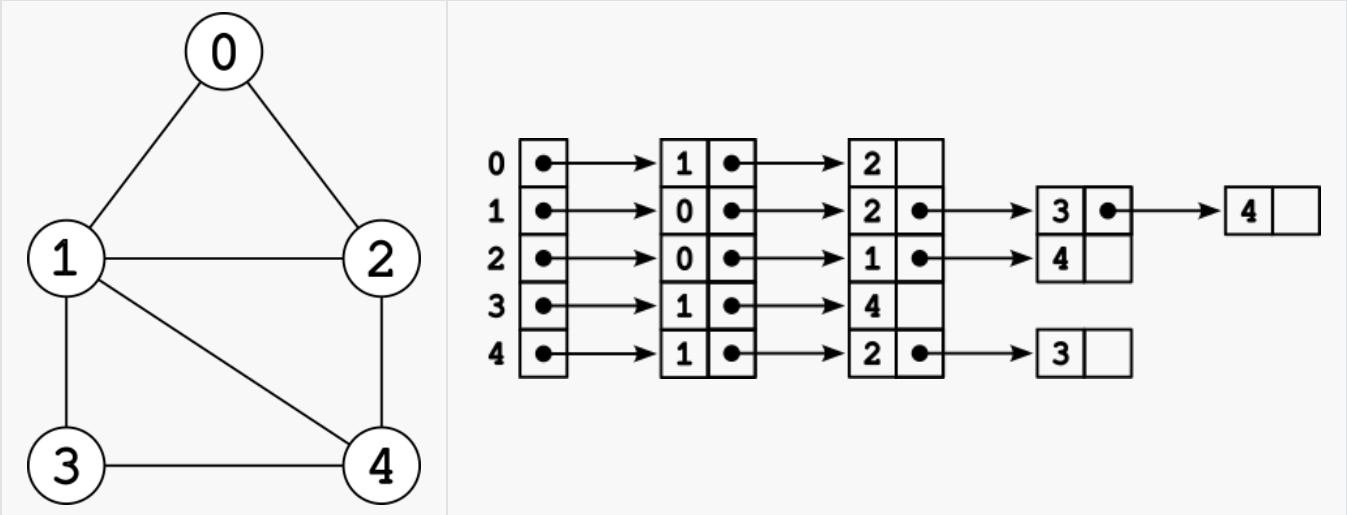
Directed and Weighted | *Adjacency Matrix*

An adjacency matrix provides for very fast lookup of edge data; however, it cannot resize efficiently. To change the size of the adjacency matrix, it costs $O(n^2)$ where n is the number vertices because we must copy a two dimensional array of size n. In other words, the adjacency matrix supports fast lookup of adjacency information, but it is inefficient if the number of nodes in graph are changing. This raises the question of whether or not there exists an alternative edge representation that is more efficient in terms of changing graph size, and the answer is to use an adjacency list.

Adjacency List

An **adjacency list**, L, uses an array of lists to consolidate edge adjacency into a single structure. In the adjacency list, the array index maps to the vertex identifier and therefore the array is of size n where n is the number of vertices.

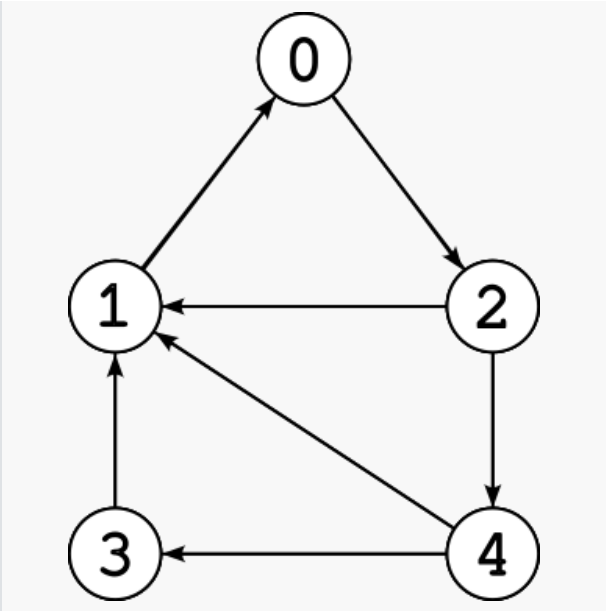
For an undirected graph, every edge associated with a vertex will appear in the list mapped to that vertex. To determine whether or not two vertices with mapped identities i and j are adjacent, either $L(i)$ or $L(j)$ can be queried and the associated list searched for the other vertex.



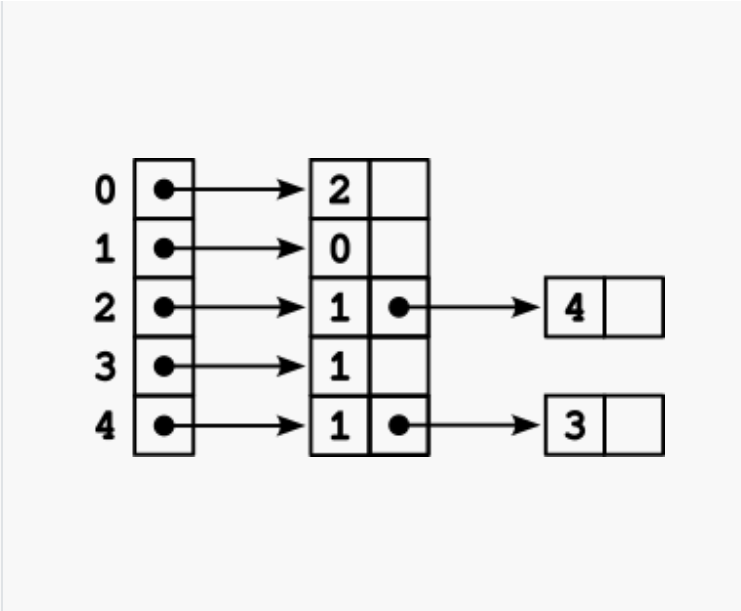
Undirected | *Adjacency List*

For example, in the above figure, the vertices adjacent to vertex 4 are 1, 2, and 3 because all of these indices exist in $L(4)$. Alternatively, we can see that vertex 4 is adjacent to only vertices 1, 2, and 3 because 4 appears in $L(1)$, $L(2)$, and $L(3)$ and no others.

For a directed graph, a minimal adjacency list can represent the directed edges by storing the terminal vertex identifiers in each vertex list.



Directed

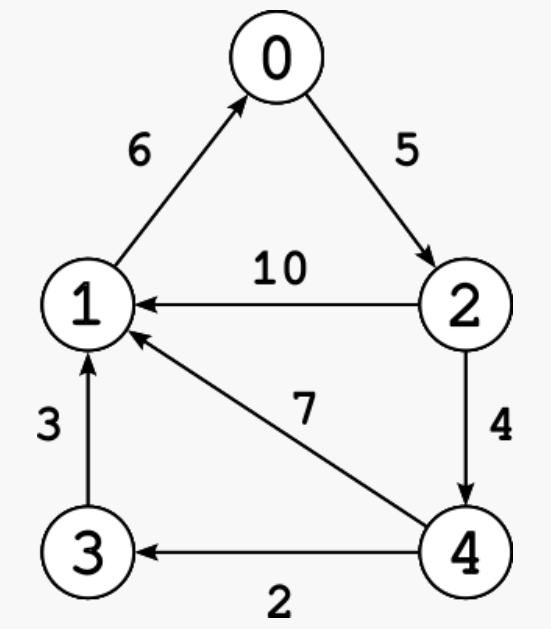


Adjacency List

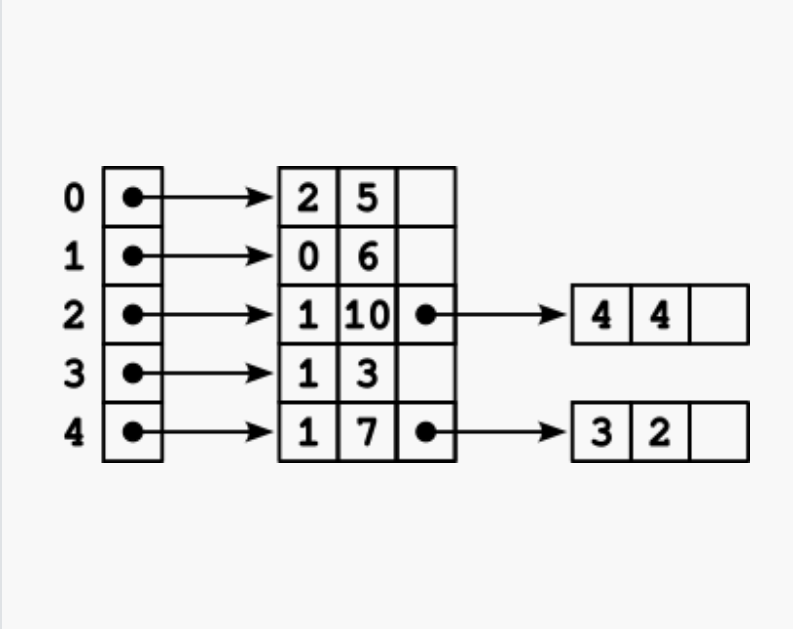
For example, in the above figure, the vertices that terminate the outgoing edges from vertex 4 are 1 and 3 because 1 and 3 are in $L(4)$. The weakness of this approach is that it is difficult to determine which vertices lead to a specific vertex without searching all vertex lists; however, this could be resolved either by maintaining two adjacency lists, one representing incoming edges and the other representing outgoing edges, or by using a vertex structure in each node of the adjacency list with a property that discriminates between incoming and outgoing edges.

```
class DirectedAdjacency {
    int vertex_id;    // vertex index/identifier
    boolean outgoing;  // if not outgoing, must be incoming
}
```

In order to maintain weights for weighted graphs, the vertex structure in each node of the adjacency list must include weights for each edge.



Directed and Weighted



Adjacency List

```
class WeightedAdjacency {
    int vertex_id;    // vertex index/identifier
    int weight;     // weight of the edge
    ...         // other properties such as direction
}
```

An adjacency list is much more object oriented and may be a more comfortable representation for today's programmers; however, adjacency queries are slower than the adjacency matrix since the matrix can service an adjacency query in constant time while the worst-case efficiency for an adjacency query in a adjacency list is linear time in the number of vertices because each list can have every vertex in it and search in a linked list is at best linear time complexity.

The advantage of an adjacency list is that it can grow more efficiently than the adjacency matrix as insert for the adjacency list in the worst case is linear time in the number of vertices instead of the quadratic time for the adjacency matrix. While linked lists can support fast append, there is likely a uniqueness constraint limiting the number of times a vertex can appear in a specific list to one, so a given list still needs to be searched for the existence of the edge before one can be inserted into a specific linked list.

Overall, an adjacency list is a better choice if the graph is dynamically changing in terms of the number of vertices in the graph, but if a graph is well established and generally stable in the number of vertices, it is more efficient to use an adjacency matrix.

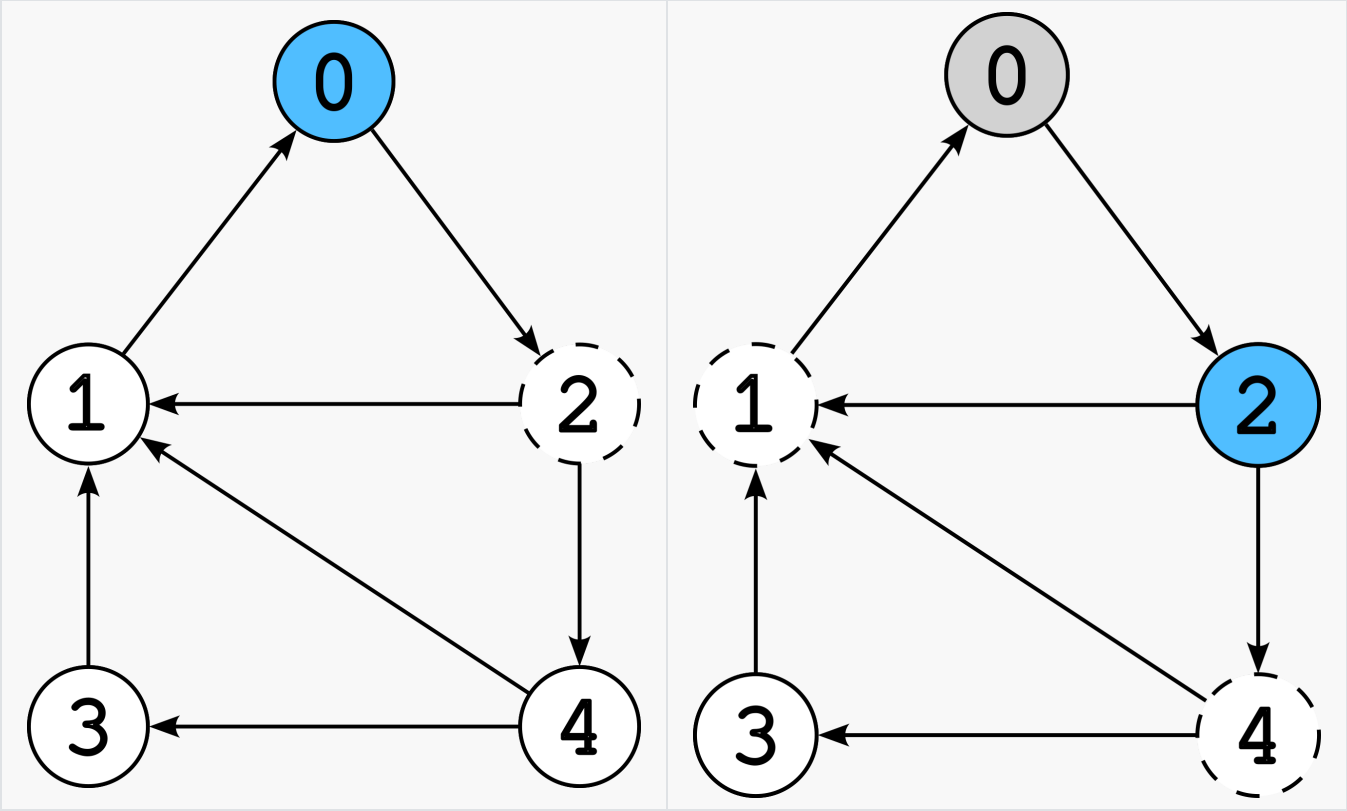
Search

Like any other data structure, one of the key algorithms for a graph is search. If search prioritizes exploring as far as possible and then backtracking when a dead end is reached, it represents a depth-first strategy. Alternatively, if search instead prioritizes exploring all adjacent vertices before further exploration, it is a breadth-first strategy. To prevent revisiting a vertex, search may require an additional "visited" property be maintained.

Visited

Unlike a tree, a graph does not have to be hierarchical and can be cyclic, so it is possible for a graph traversal to be caught in an infinite loop. Adding a **visited property** to each vertex that indicates whether or not a vertex has already been visited will give the traversal process a means to determine whether or not it has explored a vertex already and a means to escape cyclic regions of a graph. This is the same idea as leaving breadcrumbs in a maze.

The visited property can be tracked using a property associated with each vertex object in a OOP modeled graph or using an array of boolean values if a more primitive graph structure is implemented.

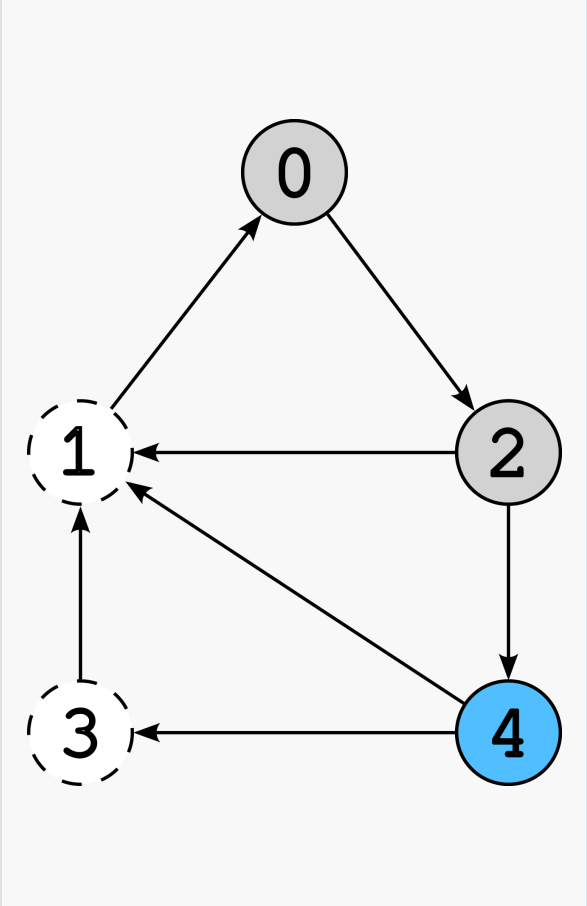


Start search at vertex 0 with vertex 2 as adjacent	Mark 0 as visited, explore the adjacent vertex 2, and determine any adjacent vertices, *i.e.* vertices 1 and 4

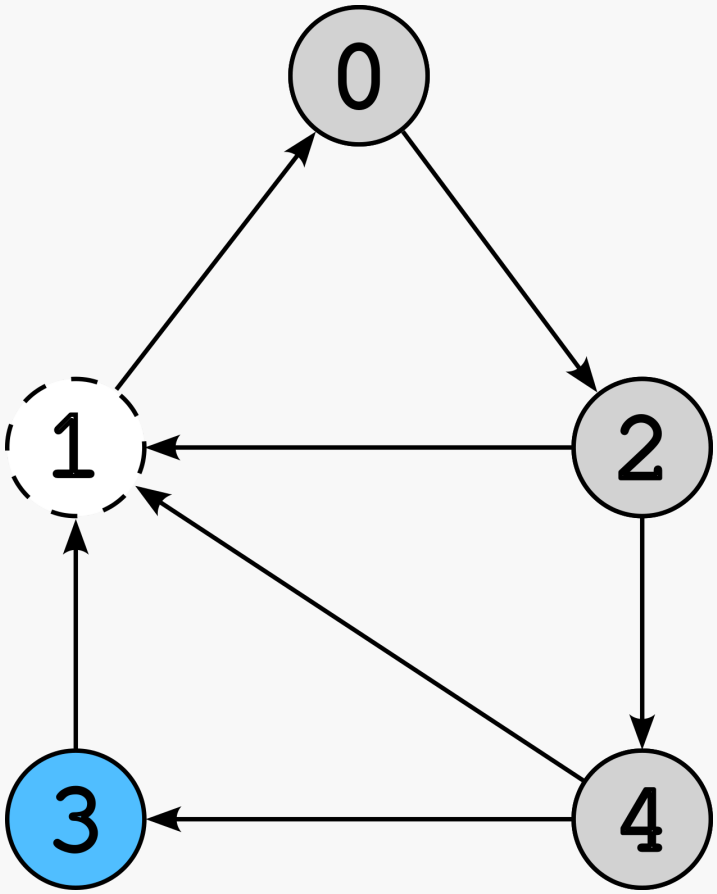
Because there are a variety of choices at a given vertex, determining which node to visit next is subject to a search policy. There are two general search policies as examined in trees: Depth-First Search and Breadth-First Search.

Depth-First Search (DFS)

Depth-First Search (DFS) in graphs is the same model as Depth-First Traversal used in Trees. In Depth-First Search, the search algorithm prioritizes exploration of all of the paths from one adjacent vertex before exploring additional paths from the current vertex.



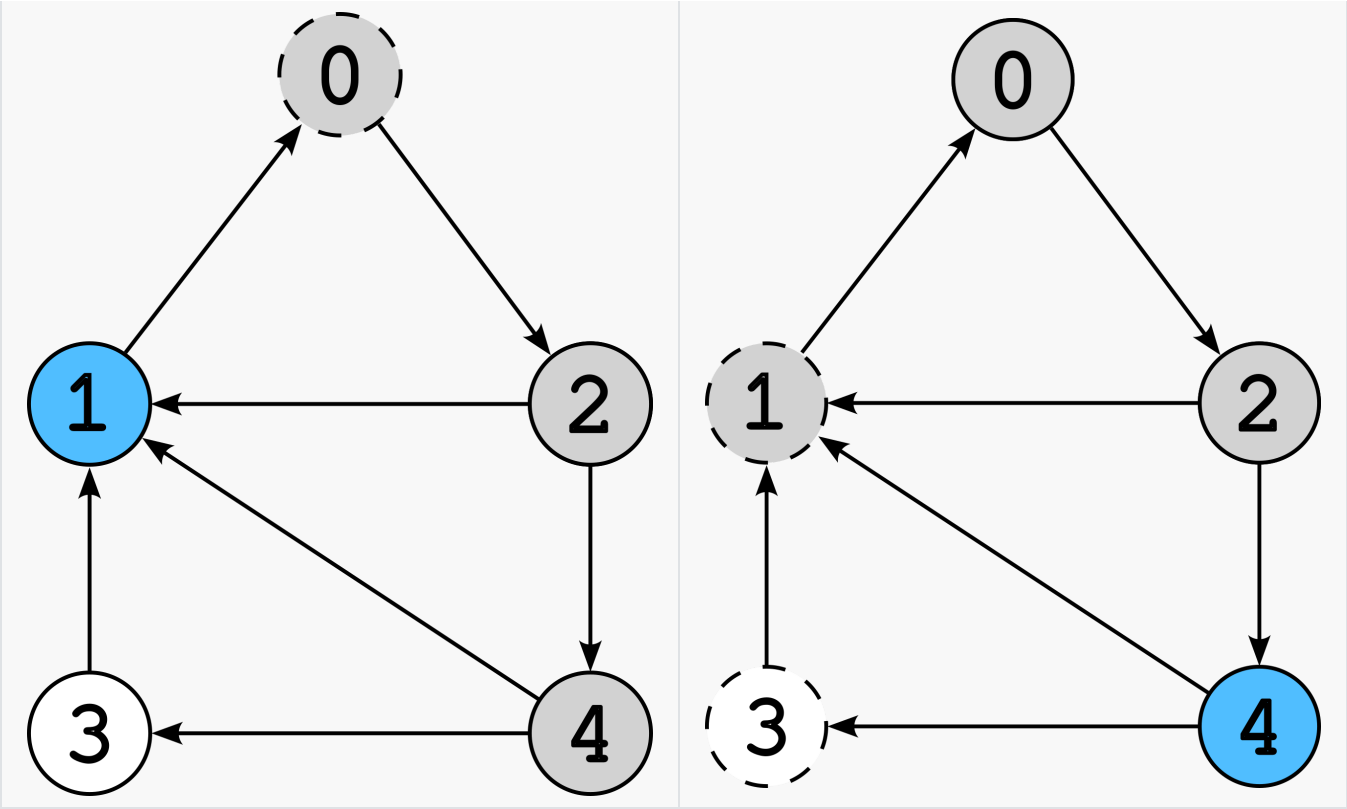
For DFS, once vertex 2 is visited, either vertex 4 or vertex 1 will be explored next. Suppose vertex 4 is selected. At vertex 4, DFS again would choose one of the adjacent vertices and continue exploration there.



Suppose vertex 3 is selected. This will ultimately lead to vertex 1 being explored leading to a cycle back to vertex 0 (or dead-end); however, if the search value was not found, DFS would backtrack to explore unexplored adjacent vertices of vertex 3, then vertex 4, then vertex 2, and then vertex 0

Breadth-First Search (BFS)

Breadth-First Search (BFS) in graphs is also the same model as Breadth-First Search traversal used in Trees. In Breadth-First Search, the search algorithm will prioritize exploration of all vertices adjacent to the current vertex before making a decision on which vertex to visit next.



For BFS, once vertex 2 is visited, both vertex 1 and vertex 4 will be explored next; however, it can only explore one at a time. Suppose vertex 1 is selected.	Continuing the scenario to the left, BFS will then explore vertex 4 after exploring vertex 1 because both vertex 1 and 4 are adjacent to vertex 2.

Shortest Path

A naive DFS or BFS will ultimately find a search value in the graph; however, the path taken to find the vertex does not necessarily reflect an optimal path especially when exploring a weighted graph. The optimal path is the path with the minimum total weight or shortest path through the graph which raises the question of how to compute this shortest path.

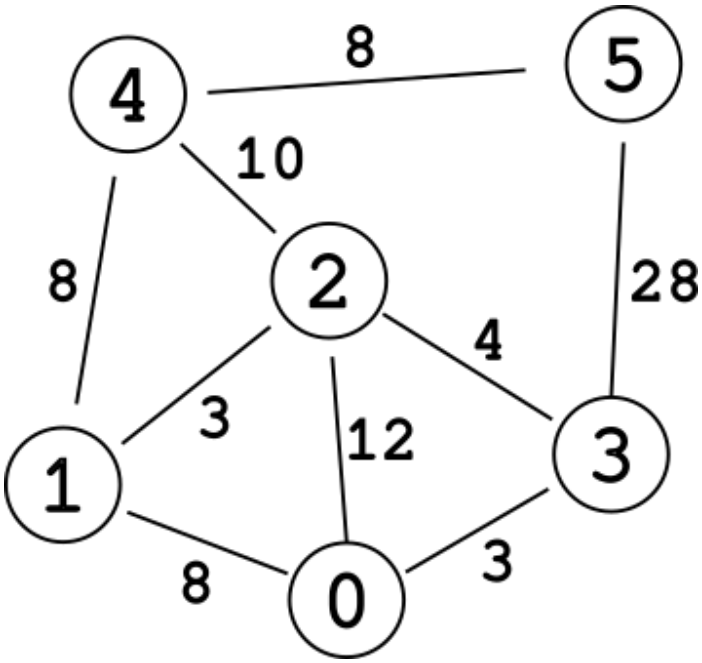
In a tree, each vertex is connected to the root by only one path, but for a more generalized graph, the number of potential paths between vertices v_{src} and v_{dest} may be quantified by a permutation that is dependent on the number of vertices in the graph. Recall from the earlier discussion of permutations, that this implies that the number of paths in a graph is generally approximated as a factorial making computation of all paths through the graph potentially computationally intensive.

In other words, the naive approach of DFS or BFS is unlikely to produce an optimal path and a brute force approach has factorial time complexity in the worst case.

The shortest path problem is generally expressed as the following: given a graph $G(V, E)$ where V is the set of vertices in G and E is the set of edges in G, compute the shortest path between vertices v_{src} and v_{dest} where v_{src} and v_{dest} are members of V and the shortest path is the path with the minimal cost among all paths from v_{src} to v_{dest}.

The Dutch computer scientist Edsger Dijkstra [1930-2002] defined an algorithm we know today as Dijkstra's Algorithm that solves the shortest path problem for any graph.

Dijkstra's Algorithm



Dijkstra's algorithm uses two arrays the same size as V to compute distance and path association: $d(i)$ and $p(i)$. $d(i)$ represents the minimum distance to the i-th vertex. $p(i)$ represents the previous vertex in the shortest path leading to the i-th vertex. The distance, d, to a vertex is recomputed whenever a neighboring vertex is visited and entries of $d(i)$ and $p(i)$ are updated when $d < d(i)$.

Let U represent the set of unvisited vertices. Marking a vertex as visited removes that vertex from the set U. When U is empty, Dijkstra's is complete and the path information can be reconstructed from $p(i)$. In other words, the shortest path can be reconstructed by starting at the destination vertex and referring to its previous node in $p(i)$ which leads to that nodes previous node, etc. leading back to the initial node:

$$v_{dest} \rightarrow p(v_{dest}) \rightarrow p(p(v_{dest})) \rightarrow \ldots \rightarrow p(v_{src}) : p(v_{src}) = \emptyset$$

If in the process of recreating this path a null, $\emptyset$, is encountered and it is not in the initial node's index, there does not exist a path from v_{src} to v_{dest}.

Let us assume function $w(i, j)$ returns the weight of the edge between v_i and v_j

In high level terms, Dijkstra's Algorithm uses the following steps:

1. Mark the distance to the source vertex, v_{src}, with a current distance of 0, $d(v_{src}) = 0$, and the rest with infinity, $d(i) = \infty : i \neq v_{src}$. Mark all node's previous as null, $p(i) = \emptyset$. Mark all nodes as unvisited, adding them to the set U.
2. Set the non-visited vertex with the smallest current distance as the current vertex v_i.
3. For each neighbor, v_j, of the current vertex in the set of neighbors, N:

   3.1 Add the distance of the current vertex with the weight of the edge between the current vertex and the

current neighbor, $d(i) + w(i, j)$.

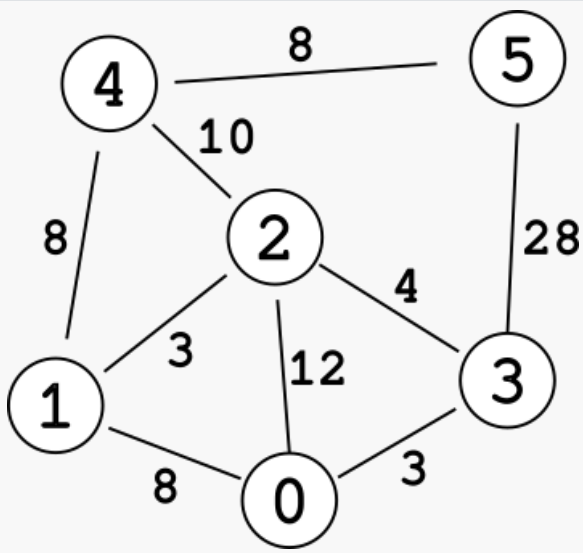
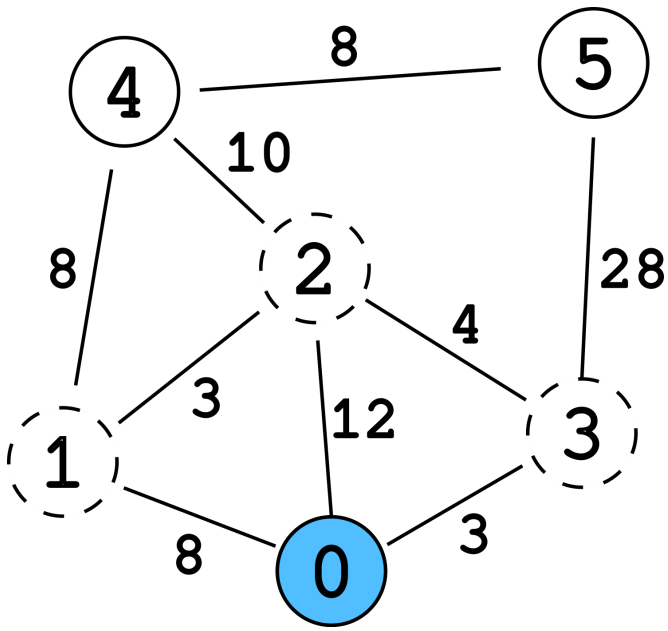
3.2 If this distance is smaller than the current distance to the current neighbor, $d(j) > d(i) + w(i, j)$:

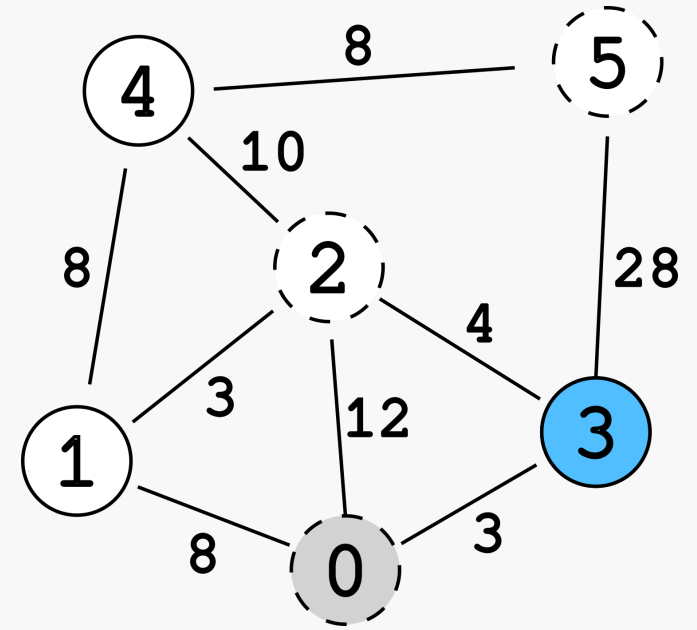
3.2.1 Set the new current distance to the neighboring vertex v_j equal to the sum of the distance to v_i and the weight of the edge between v_i and v_j, $d(j) = d(i) + w(i, j)$.

3.2.2 Update the previous vertex of the neighboring vertex to the current vertex, $p(j) = i$.

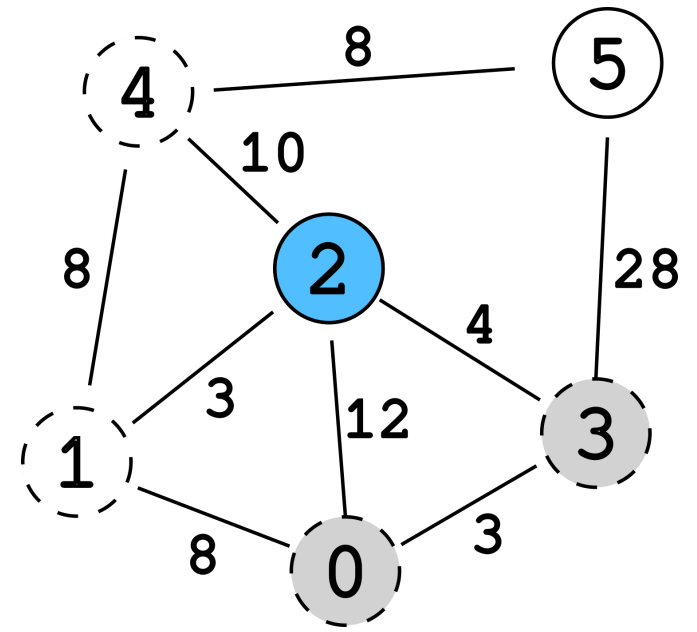
4. Mark the current vertex v_i as visited removing v_i from the set U.

5. Go to step 2 if there are any unvisited vertices.

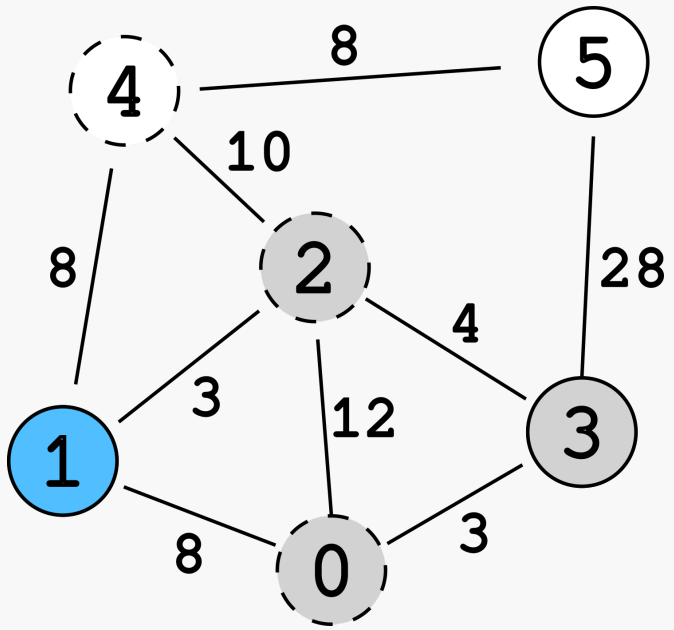
	$v_{src} = 0$ $v_{dest} = 5$ $v_i = v_{src}$ $U = \{0, 1, 2, 3, 4, 5\}$ $d(i) = [0, \infty, \infty, \infty, \infty, \infty]$ $p(i) = [\emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset]$
	$v_{src} = 0$ $v_{dest} = 5$ $v_i = 0$ $N = \{1, 2, 3\}$ $U = \{0, 1, 2, 3, 4, 5\}$ $d(i) = [0, 8, 12, 3, \infty, \infty]$ $p(i) = [\emptyset, 0, 0, 0, \emptyset, \emptyset]$



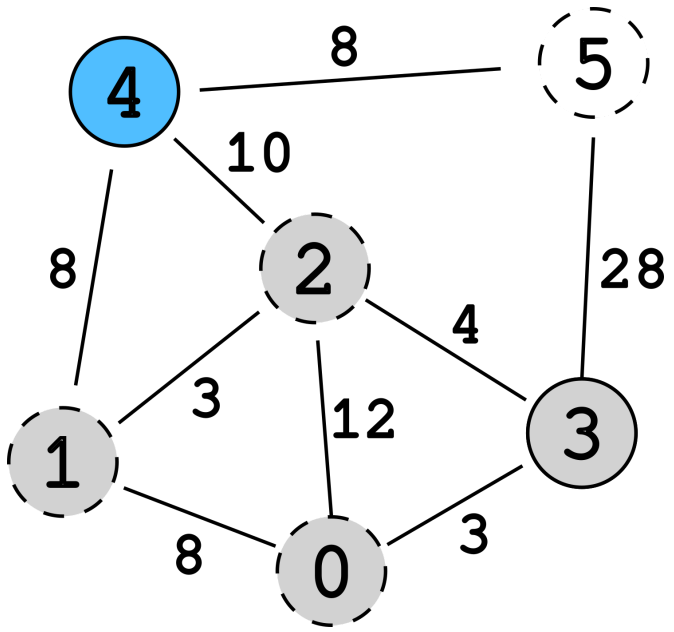
$v_{src} = 0$
$v_{dest} = 5$
$v_i = 3$
$N = \{0, 2, 5\}$
$U = \{1, 2, 3, 4, 5\}$
$d(i) = [0, 8, 7, 3, \infty, 31]$
$p(i) = [\emptyset, 0, 3, 0, \emptyset, 3]$



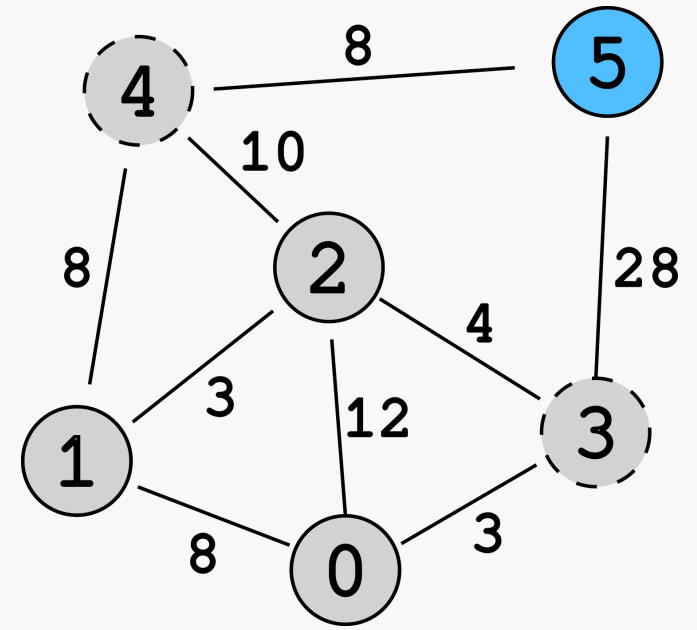
$v_{src} = 0$
$v_{dest} = 5$
$v_i = 2$
$N = \{0, 1, 3, 4\}$
$U = \{1, 2, 4, 5\}$
$d(i) = [0, 8, 7, 3, 17, 31]$
$p(i) = [\emptyset, 0, 3, 0, 2, 3]$



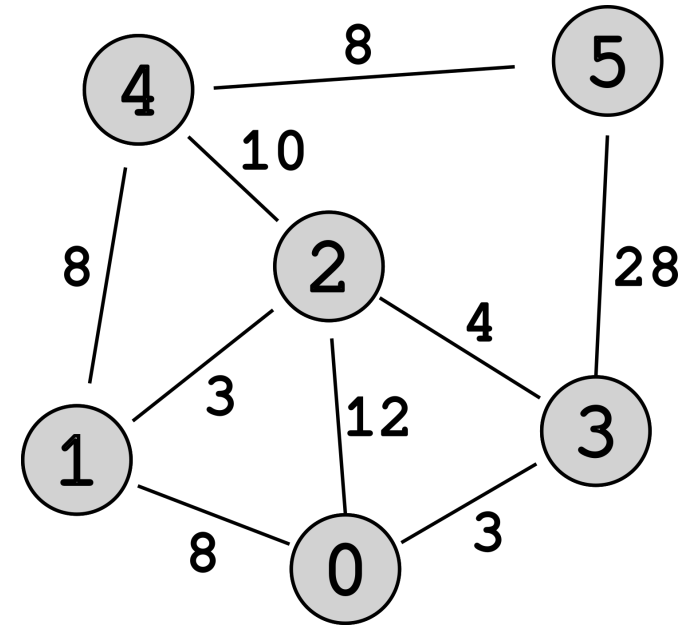
$v_{src} = 0$
$v_{dest} = 5$
$v_i = 1$
$N = \{0, 2, 4\}$
$U = \{1, 4, 5\}$
$d(i) = [0, 8, 7, 3, 16, 31]$
$p(i) = [\emptyset, 0, 3, 0, 1, 3]$



$v_{src} = 0$
$v_{dest} = 5$
$v_i = 4$
$N = \{1, 2, 5\}$
$U = \{4, 5\}$
$d(i) = [0, 8, 7, 3, 16, 24]$
$p(i) = [\emptyset, 0, 3, 0, 1, 4]$



$v_{src} = 0$
$v_{dest} = 5$
$v_i = 5$
$N = \{4, 3\}$
$U = \{5\}$
$d(i) = [0, 8, 7, 3, 16, 24]$
$p(i) = [\emptyset, 0, 3, 0, 1, 4]$



$v_{src} = 0$
$v_{dest} = 5$
$U = \{\}$
$d(i) = [0, 8, 7, 3, 16, 24]$
$p(i) = [\emptyset, 0, 3, 0, 1, 4]$
$shortestpath = v_{dest} \rightarrow p(v_{dest}) \rightarrow p(p(v_{dest})) \rightarrow \ldots \rightarrow \emptyset$
$shortestpath = 5 \rightarrow 4 \rightarrow 1 \rightarrow 0 \rightarrow \emptyset$